

agenda

- Review of what you need for the short python assignment
- Stuff you'll need for the next assignment (caesar cipher)
- Lab!

math.cos and math.sin

- These are the only two new functions today
- Each takes one argument, exactly like math.sqrt
- The argument is in radians, not degrees
- You need both for A2

```
[1] ▶ import math

x = 42
print(math.cos(x))
print(math.sin(x))
```

-0.39998531498835127
-0.9165215479156338

print() with commas

- print() takes as many values as you like, separated by commas
- It puts a space between them for you
- A quoted string prints exactly what is between the quotes; a bare name prints what is stored in it
- That is all A3 is asking for

```
[3] ▶ radius = 5      # centimeters  
     volume = (4/3) * math.pi * radius**3  
  
     print("The volume is", volume)
```

```
The volume is 523.5987755982989
```

writing a function

- `def` starts the definition, and the header ends with a colon
- The body is indented four spaces
- `word` and `n` are parameters — names used inside
- `"Spam, "` and `4` are arguments — the values you hand in
- Defining a function runs nothing. Calling it does.

```
[4] ▶ def repeat(word, n):  
      print(word * n)  
  
      repeat("Spam, ", 4)
```

Spam, Spam, Spam, Spam,

len(), + and * on strings

- len() counts the characters
- * repeats a string
- + glues two strings together
- For B1: if the last letter has to land in column 40, and the word is len(text) long — how many spaces go in front?

```
[5] ▶ print(len("Monty"))  
      print(" " * 8)  
      print(" " * 8 + "Monty")
```

Monty

for and range()

- `range(4)` starts at 0 and stops before 4
- Four values — and none of them is 4
- This is where most of the wrong output in part B comes from

```
[6] ▶ for i in range(4):  
      print(i)
```

```
0  
1  
2  
3
```

handing this in

- To hand in: File › Download › Download .py
- One .py file, problems in order, each labeled # A1, # A2, and so on

order of attack

- A1, A2, A3 — then B2 and B3 — then B1 — then B4 — then C1 and C2
- B1 is harder than B2 and B3. Do not let it stall you; come back to it.
- Write one problem, run it, then write the next. Not all nine and then run.
- Stuck? Ask!
- Book sections: A1 and A2 in 2.11.3, B1 through B4 in 3.11, C2 in 3.11.5

new tools for the caesar cipher

review: reaching one character: []

- Square brackets pull a single character out of a string
- The number inside is called the **index**
- `word[0]` is the first character

```
1 word = "python"
2
3 print(word[0])      # p
4 print(word[1])      # y
5 print(word[2])      # t
```

ord() and chr()

- Every character has a number associated with it
- ord() gives you the number; chr() gives you the character back
- They undo each other — chr(ord("a")) is just "a"
- "a" is 97 and "A" is 65. Upper and lower case are different numbers, which is why Part A handles them separately.

```
[1] ▶ print(ord('a'))  
      print(ord('A'))  
  
      print(chr(97))  
      print(chr(65))
```

97

65

a

A

shifting one letter

- The pattern for the whole assignment: letter to number, do arithmetic, number back to letter
- It works fine until you run off the end of the alphabet
- "}" is not a letter — the alphabet ran out
- Fixing that is what Part A is asking for

```
[2] ▶ letter = 'c'

print(ord(letter))
print(ord(letter) + 3)
print(chr(ord(letter) + 3))

print(chr(ord('z') + 3))
```

```
99
102
f
}
```

a really big hint

%

letter.isupper()

- A new kind of call: the dot goes after the value, not around it
- You have been writing `len(letter)`. This is `letter.isupper()`.
- Both come back `False` for anything that is not a letter
- That is how you spot the spaces and punctuation Part A tells you to leave alone

```
[4] ▶ letter = 'z'

print(letter.isupper())
print(letter.islower())

print('!'.isupper())
print('!'.islower())
```

```
True
False
False
False
```

lab!